JavaScript Master Notes

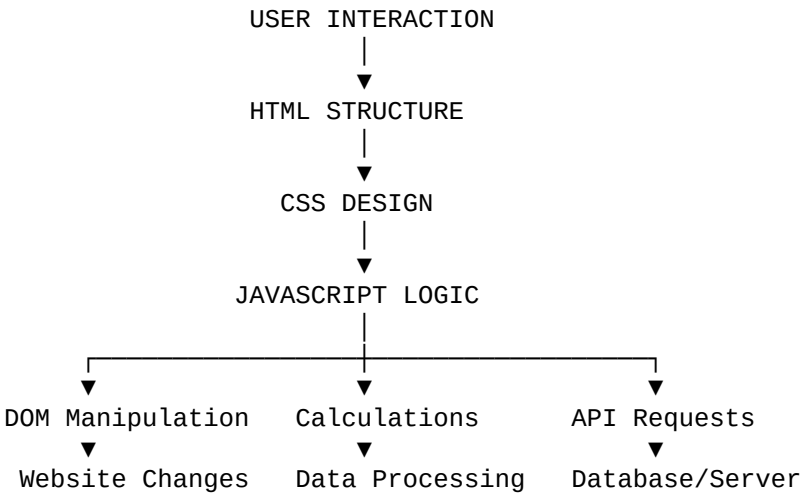
What is JavaScript?

JavaScript is a **high-level**, **interpreted**, **single-threaded** programming language mainly used to make websites interactive.

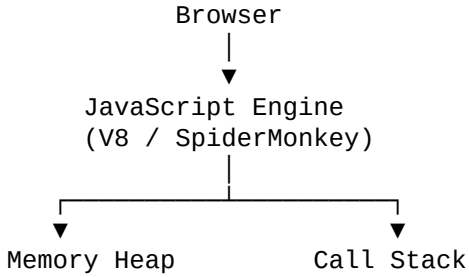
JavaScript can:

- Update website content dynamically
- Create animations
- Handle forms
- Communicate with servers
- Build full stack applications
- Create games
- Create mobile apps
- Create backend APIs

Complete JavaScript Flow Diagram



How JavaScript Runs in Browser



JavaScript Engine

Browsers use engines to run JS.

Browser	Engine
Chrome	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

---

1. Variables

Variables store values in memory.

Real Life Analogy

```
Bottle Label = Variable Name
Water Inside = Value
```

Example:

```
let name = "Akshat";
```

Memory Diagram

```
Variable Name          Stored Value
-----------------------------------
name                   "Akshat"
age                    25
isAdmin                true
```

Variable Declaration Process

```
let age = 25;
```

Steps:

```
1. Memory created
2. Variable name assigned
3. Value stored
```

---

2. Data Types

Data types define the kind of data.

Two Main Categories

```
Data Types
|
```

```
├── Primitive
│    ├── Number
│    ├── String
│    ├── Boolean
│    ├── Undefined
│    ├── Null
│    ├── Symbol
│    └── BigInt
│
└── Non Primitive
     ├── Array
     ├── Object
     └── Function
```

Primitive Data Types

Stored directly in stack memory.

Stack Memory Diagram

```
STACK MEMORY
-----------------
a = 10
b = true
name = "Akshat"
```

Non Primitive Data Types

Stored in heap memory.

Heap Memory Diagram

```
STACK                  HEAP
------                 ----------------
arr ──────────────────► [1,2,3,4]

obj ──────────────────► {
                          name:"Akshat"
                        }
```

Number

```
let age = 25;
let price = 99.99;
```

Everything numeric is:

```
console.log(typeof age);
```

Output:

```
number
```

String

```
let name = "Akshat";
```

Strings are sequence of characters.

String Memory Diagram

```
"A K S H A T"
 0 1 2 3 4 5
```

Access:

```
console.log(name[2]);
```

Output:

```
s
```

Boolean

Used in conditions.

```
let isLoggedIn = true;
```

Undefined

Declared but no value assigned.

```
let a;
console.log(a);
```

Null

Intentional empty value.

```
let user = null;
```

Difference Between Undefined and Null

Undefined	Null
JS gives automatically	Developer assigns
Means no value assigned	Means intentionally empty

Arrays

Stores multiple values.

Array Diagram

```
let arr = [10,20,30,40];
```

```
INDEX     VALUE
0  ──────► 10
1  ──────► 20
2  ──────► 30
3  ──────► 40
```

Accessing Array Elements

```
console.log(arr[2]);
```

Output:

```
30
```

Why Array Index Starts from 0?

Because memory address calculation becomes faster.

Formula:

```
Address = Base Address + (Index × Size)
```

Objects

Objects store key-value pairs.

Object Diagram

```
let student = {
  name: "Akshat",
  age: 25
};
```

```
OBJECT
│
├── name : "Akshat"
└── age  : 25
```

Nested Object

```
let student = {
  name:"Akshat",
  marks:[90,80,70],
  address:{
    city:"Delhi",
    state:"Delhi"
  }
};
```

Nested Structure Diagram

```
student
│
├── name
├── marks
│     ├── 90
│     ├── 80
│     └── 70
│
└── address
      ├── city
      └── state
```

---

3. Operators

Operators perform operations.

Arithmetic Operators

```
+   Addition
-   Subtraction
*   Multiplication
/   Division
%   Modulus
```

Modulus Operator

Returns remainder.

```
console.log(10 % 3);
```

Output:

```
1
```

Why Modulus is Important?

Used for:

- Even/odd
- Cyclic operations
- Pagination
- Rotations

Assignment Operators

```
let a = 10;
```

```
a += 5;
```

Internally:

```
a = a + 5;
```

Comparison Operators

Loose Equality (==)

Checks value only.

```
console.log("5" == 5);
```

Steps:

```
"5" → converted to number → 5
5 == 5 → true
```

Strict Equality (===)

Checks:

- Value
- Data type

```
console.log("5" === 5);
```

Output:

```
false
```

Deep Comparison Diagram

```
==
│
└── Value only

===
│
├── Value
└── Data Type
```

Logical Operators

AND (&&)

All conditions must be true.

```
TRUE && TRUE = TRUE
```

```
TRUE && FALSE = FALSE
```

OR (||)

At least one true.

```
FALSE || TRUE = TRUE
```

NOT (!)

Reverses value.

```
!true = false
!false = true
```

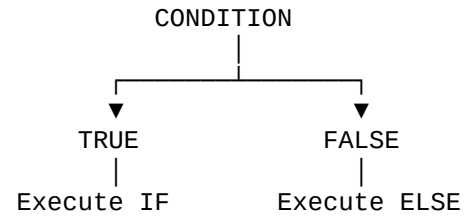
---

4. Control Flow

Controls execution flow.

---

if else Diagram



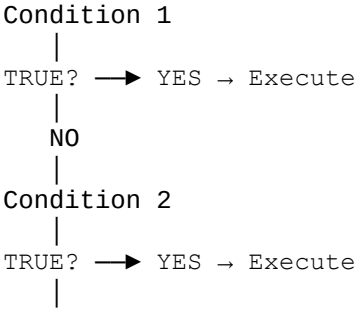
Example

```
let age = 20;

if(age >= 18){
  console.log("Adult");
}
else{
  console.log("Minor");
}
```

---

else if Ladder Flow



```
   NO
   |
Default Else
```

Switch Statement

Better for multiple exact cases.

Switch Internal Flow

```
switch(expression)
       |
       ▼
compare with case values
       |
matching case executes
```

Example

```
let day = 2;

switch(day){

  case 1:
    console.log("Monday");
    break;

  case 2:
    console.log("Tuesday");
    break;

  default:
    console.log("Invalid");
}
```

Why break is Important?

Without break:

```
Execution continues to next cases
```

This is called:

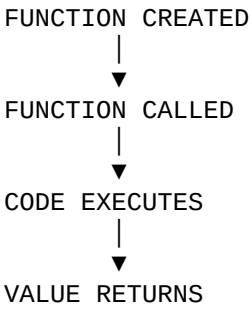
Fall Through

---

5. Functions

Functions are reusable blocks of code.

Function Flow



Function Declaration

```
function greet(){
  console.log("Hello");
}
```

Function Call

```
greet();
```

Parameters vs Arguments

```
function add(a,b){
  return a+b;
}

add(2,3);
```

Term	Meaning
Parameters	Variables in function
Arguments	Actual passed values

Return Keyword

Stops function execution and sends value back.

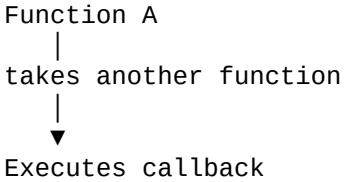
```
function add(a,b){
  return a+b;
}
```

Higher Order Functions

Functions that:

- take function as argument
- return function

Higher Order Diagram



Callback Example

```
function greet(name){
  console.log(name);
}

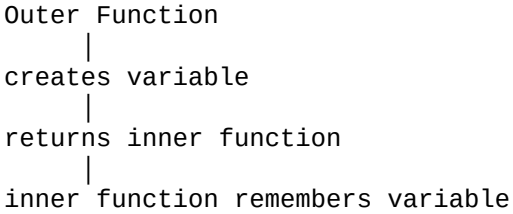
function test(func){
  func("Akshat");
}

test(greet);
```

Closures

Closure remembers outer variables even after execution.

Closure Diagram



Example

```
function outer(){

  let count = 0;

  return function(){

    count++;

    console.log(count);

  }

}

let x = outer();

x();
x();
```

Output:

```
1
2
```

6. Loops

Loops repeat code.

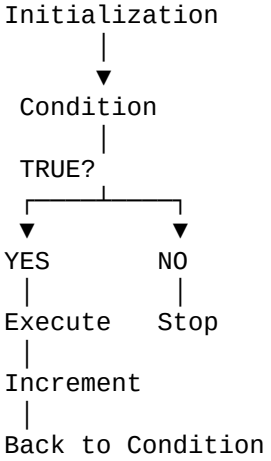
---

for Loop Structure

```
for(initialization; condition; increment){

}
```

Flow Diagram



Example

```
for(let i=1;i<=5;i++){
  console.log(i);
}
```

---

while Loop

```
let i = 1;

while(i<=5){

  console.log(i);

  i++;

}
```

---

do while Loop

Runs once even if condition false.

```
do{
```

```
}while(condition);
```

Nested Loops

Loop inside another loop.

Nested Loop Diagram

```
Outer Loop
   │
   ├── Inner Loop
   ├── Inner Loop
   └── Inner Loop
```

Multiplication Table Logic

```
for(let i=1;i<=10;i++){

  console.log(`5 x ${i} = ${5*i}`);

}
```

---

7. Scope

Scope decides accessibility.

Scope Types

```
Scope
│
├── Global
├── Function
└── Block
```

Global Scope

Accessible everywhere.

```
let a = 10;
```

Function Scope

```
function test(){

  var x = 10;

}
```

Only inside function.

Block Scope

```
{
  let x = 10;
}
```

Only inside block.

---

var vs let vs const

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No
Hoisted	Yes	Yes	Yes

---

8. Hoisting

JavaScript moves declarations to top before execution.

Hoisting Diagram

```
CODE WRITTEN
      |
      ▼
MEMORY CREATION PHASE
      |
      ▼
EXECUTION PHASE
```

---

var Hoisting

```
console.log(a);

var a = 10;
```

Internally:

```
var a;

console.log(a);

a = 10;
```

Output:

```
undefined
```

---

let/const Hoisting

```
console.log(a);

let a = 10;
```

Output:

```
ReferenceError
```

Temporal Dead Zone (TDZ)

```
Memory allocated
      |
Cannot access variable
      |
Until declaration line reached
```

---

9. Array Methods

---

push()

Adds at end.

```
arr.push(100);
```

---

pop()

Removes last.

```
arr.pop();
```

---

shift()

Removes first.

```
arr.shift();
```

---

unshift()

Adds at beginning.

```
arr.unshift(10);
```

---

slice()

Does NOT modify original array.

```
let newArr = arr.slice(1,4);
```

---

splice()

Modifies original array.

```
arr.splice(1,2,100);
```

---

slice vs splice

slice	splice
Non destructive	Destructive
Returns copy	Modifies original

---

10. Higher Order Array Methods

---

filter()

Returns matching elements.

Filter Flow

```
Array Element
      |
Condition Check
      |
TRUE?
 |         |
YES       NO
 |         |
Added    Ignored
```

Example

```
let result = products.filter(function(product){

  return product.price > 50000;

});
```

---

find()

Returns first matching element only.

```
let result = products.find(function(product){

  return product.brand === "Samsung";

});
```

---

map()

Transforms every element.

```
let prices = products.map(function(product){

  return product.price - 10000;

});
```

Map Internal Flow

```
Original Array
      |
Apply transformation
      |
Create new array
```

---

forEach()

Loops through array.

```
products.forEach(function(product){

  console.log(product.brand);

});
```

Important Difference

Method	Returns
forEach	undefined
map	new array

---

reduce()

Reduces array to single value.

Reduce Flow Diagram

```
Accumulator = 0

0 + 10 = 10
10 + 20 = 30
30 + 40 = 70
```

Example

```
let total = products.reduce(function(acc, product){
```

```
  return acc + product.price;

},0);
```

Real Life Uses of reduce

- Total cart price
- Total marks
- Counting frequency
- Flatten arrays

---

11. DOM Manipulation

DOM means:

Document Object Model

Browser converts HTML into object structure.

DOM Tree Diagram

```
document
   │
   └── html
        │
        ├── head
        │
        └── body
             │
             ├── h1
             └── p
```

Selecting Elements

```
document.getElementById("demo");
```

Changing Content

```
document.getElementById("demo").innerHTML = "Hello";
```

---

12. Event Loop

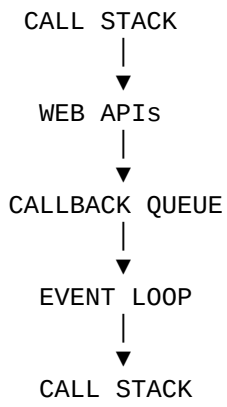
JavaScript is single threaded.

Can do:

- one task at a time

But async operations handled using event loop.

Event Loop Architecture



Example

```
console.log("Start");

setTimeout(function(){
  console.log("Timeout");
},0);

console.log("End");
```

Output:

```
Start
End
Timeout
```

Why?
Because timeout goes to Web API first.

---

13. Promises

Promises handle asynchronous operations.

Promise States

```
Promise
│
├── Pending
├── Resolved
└── Rejected
```

Promise Example

```
let promise = new Promise(function(resolve,reject){
```

```
  let success = true;

  if(success){
    resolve("Data received");
  }
  else{
    reject("Error");
  }

});

promise
.then(function(data){

  console.log(data);

})
.catch(function(error){

  console.log(error);

});
```

Async Await

Cleaner way to handle promises.

```
async function test(){

  let data = await fetch("url");

}
```

JavaScript Execution Context

Phases

```
1. Memory Creation Phase
2. Code Execution Phase
```

Example

```
console.log(a);

var a = 10;

function test(){
  console.log("Hello");
}
```

Reference Links

Official JavaScript Documentation

- MDN JavaScript Docs
- JavaScript.info
- W3Schools JavaScript Tutorial
- ECMAScript Official Docs

Best YouTube Channels

- CodeWithHarry
- Akshay Saini Namaste JavaScript
- Hitesh Choudhary

Best Practice Roadmap

```
JavaScript Basics
      |
      ▼
Functions + Arrays
      |
      ▼
Objects + DOM
      |
      ▼
Async JavaScript
      |
      ▼
ES6+
      |
      ▼
Projects
      |
      ▼
React / Node.js
```

Mini Projects for Practice

1. Calculator
2. Todo App
3. Weather App
4. Quiz App
5. Ecommerce Cart
6. Expense Tracker
7. Notes App
8. Chat App
9. LMS System

10. Banking Dashboard

Important Interview Topics

```
Closures
Hoisting
Event Loop
Promises
Async Await
Call Stack
this keyword
Prototype
Inheritance
Execution Context
Array Methods
Debouncing
Throttling
Currying
```

Deep Understanding Formula

```
Syntax + Internal Working + Real Example + Practice
= Mastery
```